

Cómo se construyó paso a paso

Tutorial práctico para comprender, mantener y ampliar la música y los efectos de sonido de EduCore.



Objetivo: que la música y los efectos tengan controles independientes, que todos los efectos respeten el volumen elegido en Configuración y que añadir un sonido nuevo sea sencillo.

Contenido

- Arquitectura y archivos de sonido
- Catálogo central en efectos.py
- Volúmenes independientes y persistencia en audio.py
- Conexión con Configuración y con los eventos del nivel
- Ejemplo completo para añadir un efecto nuevo

1. La arquitectura general

El sistema se dividió en responsabilidades pequeñas. El código que detecta una acción no necesita conocer rutas, extensiones ni cálculos de volumen. Solo solicita un efecto por su nombre.

```
Evento del juego
-> efectos.reproducir_salto()
-> GestorEfectos.reproducir(SALTO)
-> gestor_audio.reproducir_efecto(...)
-> pygame.mixer.Channel.set_volume(...)
```

Esta separación tiene tres ventajas:

- Una única configuración para cada efecto.
- Todos los sonidos respetan automáticamente el volumen grupal.
- El motor queda libre de rutas y objetos `pygame.mixer.Sound`.

2. Dónde viven los archivos

Los archivos se colocan en `assets/efectos/`.

Evento	Archivo
Salto	efecto_salto.ogg
Enemigo eliminado	enemigo_eliminado.ogg
Daño recibido	recibir_dano.ogg
Respuesta correcta	respuesta_correcta.ogg
Respuesta incorrecta	respuesta_incorrecta.ogg
Muerte	sonido_muerte.ogg

Se aceptan archivos `.ogg`, `.wav` y `.mp3`. Para un juego conviene usar OGG: ocupa poco espacio y funciona bien con Pygame.

3. El catálogo central de efectos

En `juego/core/efectos.py` se declararon constantes. Estas son identificadores internos, no nombres de archivos.

```
ENEMIGO_ELIMINADO = "enemigo_eliminado"
SALTO = "salto"
RESPUESTA_INCORRECTA = "respuesta_incorrecta"
RECIBIR_DANO = "recibir_dano"
RESPUESTA_CORRECTA = "respuesta_correcta"
MUERTE = "muerte"
```

Cada identificador apunta a una configuración:

```
@dataclass(frozen=True)
class ConfiguracionEfecto:
    archivo: str
    volumen: float
    tonos_respaldo: tuple[tuple[float, int], ...]
```

Campo	Qué controla
archivo	Nombre del archivo dentro de <code>assets/efectos</code> .
volumen	Intensidad individual entre 0.0 y 1.0.
tonos_respaldo	Tonos generados cuando falta el archivo.

Ejemplo: configuración de muerte

```
MUERTE: ConfiguracionEfecto(
    archivo="sonido_muerte.ogg",
    volumen=0.80,
    tonos_respaldo=((210, 120), (150, 150), (90, 240)),
),
```

El tono de respaldo evita que el juego falle si un archivo todavía no existe. Cuando agregas el archivo real y recargas, se utiliza el archivo automáticamente.

4. Cómo se carga y reproduce un efecto

El método reproducir recibe el identificador, encuentra su configuración y delega el volumen final al gestor de audio.

```
def reproducir(self, nombre: str):
    configuracion = CONFIGURACIONES.get(nombre)

    if configuracion is None:
        return None

    if not self.cargar():
        return None

    return gestor_audio.reproducir_efecto(
        self.sonidos.get(nombre),
        volumen_relativo=configuracion.volumen,
    )
```

Después se crean métodos con nombres fáciles de leer:

```
def reproducir_salto(self):
    return self.reproducir(SALTO)

def reproducir_dano(self):
    return self.reproducir(RECIBIR_DANO)

def reproducir_muerte(self):
    return self.reproducir(MUERTE)
```

Desde el resto del proyecto se usa efectos.reproducir_muerte(). Ningún nivel necesita conocer la ruta assets/efectos/sonido_muerte.ogg.

Carga inteligente

- Busca primero el nombre y extensión configurados.
- Si no lo encuentra, prueba OGG, WAV y MP3 con el mismo nombre.
- Mantiene los sonidos en memoria para no cargarlos en cada evento.
- Recarga todo si cambia la configuración del mezclador.

5. Separación entre música y efectos

El gestor de audio mantiene dos grupos independientes. Las variables antiguas continúan representando la música y se añadieron variables específicas para efectos.

```
# Música
self._volumen = 70
self._ultimo_volumen = 70
self._silenciado = False

# Efectos
self._volumen_efectos = 70
self._ultimo_volumen_efectos = 70
self._efectos_silenciados = False
```

Control	Música	Efectos
Volumen	_volumen	_volumen_efectos
Último valor	_ultimo_volumen	_ultimo_volumen_efectos
Silencio	_silenciado	_efectos_silenciados

Conversión para Pygame

La interfaz usa porcentajes de 0 a 100. Pygame usa números de 0.0 a 1.0, por lo que se divide entre 100.

```
@property
def volumen_efectos_normalizado(self) -> float:
    if self._efectos_silenciados:
        return 0.0

    return self._volumen_efectos / 100.0
```

Silenciar música ya no silencia los efectos. Silenciar efectos tampoco modifica la música. Cada botón conserva su último valor.

6. La fórmula del volumen final

El volumen de Configuración es grupal. El valor definido en efectos.py es particular para un sonido.

volumen final = volumen grupal de efectos × volumen individual

Dato	Valor
Deslizador de efectos	50% = 0.50
Volumen individual del salto	0.70
Resultado	$0.50 \times 0.70 = 0.35 = 35\%$

```
canal.set_volume(  
    self.volumen_efectos_normalizado  
    * volumen_relativo  
)
```

¿Por qué conservar dos niveles de volumen?

- El jugador puede subir o bajar todos los efectos desde Configuración.
- El desarrollador puede hacer el daño más fuerte y el salto más suave.
- La mezcla mantiene proporciones coherentes al cambiar el deslizador.

Para cambiar solo un efecto, edita volumen en CONFIGURACIONES. Para cambiar todos, usa el deslizador de efectos.

7. Guardado permanente con QSettings

Los valores se guardan con claves distintas. Así permanecen incluso después de cerrar el juego.

```
# Música
"audio/volumen"
"audio/ultimo_volumen"
"audio/silenciado"

# Efectos
"audio/volumen_efectos"
"audio/ultimo_volumen_efectos"
"audio/efectos_silenciados"
```

El proceso de guardado

- El jugador mueve un deslizador o pulsa un botón de silencio.
- El gestor actualiza la variable correspondiente.
- guardar() escribe los valores con QSettings.
- aplicar_volumen() actualiza Pygame inmediatamente.
- La próxima ejecución usa recargar() para recuperarlos.

Cambiar el valor predeterminado de 70 en el código solo afecta a usuarios sin una configuración guardada. Un valor existente tiene prioridad.

Compatibilidad con configuraciones anteriores

Cuando todavía no existen las nuevas claves de efectos, el sistema hereda el volumen antiguo. Esto evita un cambio brusco la primera vez que se ejecuta la versión nueva.

8. Conexión con la pantalla Configuración

La interfaz tiene dos deslizadores:

Control	Función
hzs_sonido	Volumen de música
hsz_efectos	Volumen grupal de efectos

```
def cambiar_volumen_efectos(self, valor: int):  
    self._establecer_volumen_efectos_gestor(valor)
```

El gestor emite señales para que la pantalla actualice el deslizador, el icono y el texto accesible del botón.

```
volumen_efectos_cambiado = QtCore.pyqtSignal(int)  
silencio_efectos_cambiado = QtCore.pyqtSignal(bool)
```

Por qué se usa QSignalBlocker

Cuando el gestor cambia un deslizador desde código, el propio deslizador podría volver a llamar al gestor. QSignalBlocker impide esa repetición mientras se actualiza el valor visual.

```
bloqueador = QtCore.QSignalBlocker(self.hsz_efectos)  
self.hsz_efectos.setValue(porcentaje_efectos)  
del bloqueador
```

Mientras Configuración está abierta desde un nivel, el motor vuelve a leer QSettings cada 50 ms. Por eso el cambio se escucha sin reiniciar el nivel.

9. Conexión con los eventos del juego

Cada evento llama a un método pequeño y descriptivo. La carga, el formato y el volumen permanecen centralizados.

```
# Al saltar
efectos.reproducir_salto()

# Al recibir daño
efectos.reproducir_dano()

# Al responder
efectos.reproducir_respuesta_correcta()
efectos.reproducir_respuesta_incorrecta()

# Al pisar un enemigo
efectos.reproducir_enemigo_eliminado()

# Al quedarse sin vidas
efectos.reproducir_muerte()
```

Caso especial: muerte

Antes, motor.py guardaba la ruta, cargaba el Sound y lo reproducía. Todo eso se retiró. Ahora motor.py solamente detecta el game over y dispara el evento:

```
if not self.sonido_muerte_reproducido:
    if pygame.mixer.get_init():
        pygame.mixer.music.stop()

    efectos.reproducir_muerte()
    self.sonido_muerte_reproducido = True
```

La variable sonido_muerte_reproducido evita que el efecto se repita en cada frame mientras continúa la pantalla de game over.

No uses sonido.play() directamente para estos eventos. Esa llamada puede omitir el volumen grupal y rompe la arquitectura común.

10. Cómo añadir un efecto nuevo

Ejemplo: reproducir un sonido cuando el jugador recoge una moneda.

Paso 1. Coloca el archivo

```
assets/efectos/recoger_moneda.ogg
```

Paso 2. Declara un identificador

```
RECOGER_MONEDA = "recoger_moneda"
```

Paso 3. Añade la configuración

```
RECOGER_MONEDA: ConfiguracionEfecto(  
    archivo="recoger_moneda.ogg",  
    volumen=0.75,  
    tonos_respaldo=((700, 70), (950, 100)),  
),
```

Paso 4. Crea el método de ayuda

```
def reproducir_recoger_moneda(self):  
    return self.reproducir(RECOGER_MONEDA)
```

Paso 5. Llama al efecto desde el evento

```
efectos.reproducir_recoger_moneda()
```

No necesitas volver a programar volumen, silencio, guardado ni compatibilidad de formatos. El nuevo efecto hereda todo el sistema.

11. Ajustes y errores comunes

Cambiar el volumen de un solo efecto

```
SALTO: ConfiguracionEfecto(  
    archivo="efecto_salto.ogg",  
    volumen=0.55, # Más suave  
    tonos_respaldo=((420, 65), (620, 75)),  
),
```

Errores frecuentes

Problema	Revisión
No suena	Comprueba nombre, carpeta y extensión.
Suena muy bajo	Revisa volumen grupal y volumen individual.
No responde al deslizador	Asegúrate de usar <code>gestor_audio.reproducir_efecto</code> .
Se repite muchas veces	Protege el evento con una bandera o cooldown.
Cambiaste 70 y no ocurre nada	Ya existe un valor guardado en <code>QSettings</code> .
El botón cambia solo	Actualiza el widget usando <code>QSignalBlocker</code> .

Lista de comprobación

- El archivo está dentro de `assets/efectos`.
- Existe una constante para el identificador.
- `CONFIGURACIONES` contiene `archivo`, `volumen` y `respaldo`.
- Existe un método `reproducir_nombre_del_efecto`.
- El evento llama a efectos, no directamente a `Sound.play()`.
- El volumen individual está entre 0.0 y 1.0.
- Se probó con música y efectos en porcentajes diferentes.

12. Mapa de archivos y resumen final

Archivo	Responsabilidad
juego/core/efectos.py	Catálogo, carga y métodos de cada efecto.
juego/sistemas/audio.py	Volúmenes, silencios, guardado y Pygame.
CODIGOS/Ajustes.py	Deslizadores, botones y sincronización visual.
juego/core/motor.py	Eventos de salto, daño, respuesta y muerte.
juego/core/juego.py	Evento de enemigo eliminado.
assets/efectos/	Archivos OGG, WAV o MP3.

Idea central: los eventos solo indican qué ocurrió. efectos.py decide qué sonido corresponde y audio.py decide con qué volumen debe reproducirse.

Resultado obtenido

- Música y efectos con porcentajes independientes.
- Botones de silencio independientes.
- Valores conservados al cerrar el programa.
- Volumen grupal combinado con volumen particular.
- Muerte centralizada junto con los demás efectos.
- Sistema preparado para añadir sonidos sin duplicar lógica.

Para practicar, intenta añadir el efecto recoger_moneda y pruébalo con efectos al 30% y música al 80%. Si ambos se mantienen independientes, la integración es correcta.